

Brute Force - Sorting Algorithms

1. Selection Sort

Question 1: Trophy Ranking System

Approach: Instead of sorting the entire array, repeatedly scan the remaining unprocessed portion to find the maximum, move it to the front, and shrink the unprocessed region. Stop after 5 iterations since only the top 5 scores are required. This avoids the cost of sorting all 50 values fully.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(k \times n)$	Only k passes are made instead of n, each scanning the remaining n-i elements to find the max.
Space	$O(1)$	Sorting is done in place; only a copy of the input array is used for safety.

Compiler Output / Test Cases:

```
assert K_scores([72,88,65,90,77,95,60,83,91,68], 5) ==  
[95,91,90,88,83]  
assert p_k_scores([5,3,1], 5) == [5,3,1] # fewer than k items  
assert top_k_scores([], 3) == [] # empty input  
print('All test cases passed!')
```

Question 2: Memory-Constrained Embedded Device

Approach: Use the standard selection sort algorithm, but only perform a swap when the found minimum is not already in the correct position. Track the number of swaps performed and compare it against n-1 (the theoretical maximum) to demonstrate the low write cost.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n^2)$	Two nested loops compare every pair of elements regardless of input order.
Writes	$\leq n-1$	A swap only occurs when the minimum found is not already in place, bounding total memory writes.

Compiler Output / Test Cases:

```
res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])
assert sw <= len(res) - 1 # write bound holds
res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])
assert sw2 == 0 # already sorted -> zero swaps
print('All test cases passed!')
```

Question 3: Library Book Reordering

Approach: Represent each book as its ID in an array. Apply selection sort, incrementing a move counter only on an actual swap (a physical book relocation), while comparisons (label checks) are unrestricted since they are cheap.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n^2)$	Comparisons dominate the runtime; for $n=20$ this is 190 comparisons.
Physical Moves	$\leq n-1$	Each shelf position is corrected with at most one physical swap.

Compiler Output / Test Cases:

```
ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])
assert moves <= len(ordered) - 1
ordered2, moves2 = reorder_shelf([100, 200, 300]) # already sorted
assert moves2 == 0
print('All test cases passed!')
```

Question 4: Contest Prize Distribution

Approach: Treat this as selection sort in descending order: for each position, scan the remaining participants to find the one with the highest score, swap them into place, and print their name as the next-ranked winner.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n^2)$	Each of the n ranking positions requires a full scan of the remaining participants.
Space	$O(1)$ extra	Ranking is produced in place using swaps; the

		output list just records the order.
--	--	-------------------------------------

Compiler Output / Test Cases:

```
ranking = distribute_prizes([('Asha', 88), ('Ravi', 95), ('Meera', 79), ('Dev', 95)])
scores_only = [p[1] for p in ranking]
assert scores_only == sorted(scores_only, reverse=True) #
descending order
assert ranking[0][1] == 95 # top score first
print('All test cases passed!')
```

Question 5: Small Dataset in a Mobile App

Approach: For very small n , the $O(n^2)$ cost of selection sort is negligible in absolute terms (a handful of microseconds), and its simplicity and low swap count make it a reasonable, easy-to-maintain choice rather than pulling in a general-purpose sort for 8-10 items.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n^2)$ $n \leq 10$	For such small n , absolute execution time is in the microsecond range, so complexity class is not a practical concern.
Space	$O(1)$	In-place sort, no extra memory beyond the working copy.

Compiler Output / Test Cases:

```
assert selection_sort([499, 129, 899, 45, 275, 60, 310, 150]) ==
sorted([499, 129, 899, 45, 275, 60, 310, 150])
assert selection_sort([]) == []
assert selection_sort([7]) == [7]
print('All test cases passed!')
```

2. Bubble Sort

Question 1: Exam Result Slip Correction

Approach: Run bubble sort with a 'swapped' flag that tracks whether any swap occurred in a pass. If a pass completes with no swaps, the list is already sorted and the algorithm exits early - ideal for nearly sorted data like this.

Complexity Analysis:

Aspect	Complexity	Explanation
--------	------------	-------------

Best Case	$O(n)$	One pass detects a fully sorted array and exits when no swaps occur.
This Scenario	$\approx O(n)$	Only 2-3 misplaced roll numbers means very few passes are needed before the flag stays false.

Compiler Output / Test Cases:

```
sorted_rolls, passes =
optimized_bubble_sort([101,102,104,103,105,107,106,108])
assert sorted_rolls == sorted([101,102,104,103, 105, 107, 106,
108])
assert passes < 8 # fewer than full n passes
sorted_ok, passes_ok = optimized_bubble_sort([1,2,3,4,5]) #
already sorted
assert passes_ok == 1 # exits after first pass
print('All test cases passed!')
```

Question 2: Traffic Signal Priority Queue

Approach: Assign a numeric priority to each vehicle type. Whenever a vehicle joins the queue, append it and re-run bubble sort on the (small) queue to restore priority order. Since the queue is small and rarely has more than a handful of vehicles, bubble sort's $O(n^2)$ cost is negligible, and its simplicity makes it easy to reason about and verify in a safety-critical system.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n^2)$	Re-sorting the whole small queue on every arrival is acceptable since n stays very small.
Trade-off	Simplicity vs. efficiency	Bubble sort is easy to verify for correctness in a safety-critical setting, which outweighs its inefficiency at this scale.

Compiler Output / Test Cases:

```
queue = ['car', 'car', 'bus']
queue.append('ambulance')
result = bubble_sort_queue(queue)
assert result == ['ambulance', 'bus', 'car', 'car']
assert bubble_sort_queue(['ambulance']) == ['ambulance'] # single
vehicle
print('All test cases passed!')
```

Question 3: Bubble Sort Visualization Tool

Approach: Modify standard bubble sort so that after each outer-loop pass, the current state of the array is stored (or printed) as a frame. These frames can then be fed into an animation or step-through UI.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n^2)$	Same as standard bubble sort; storing frames adds $O(n)$ work per pass.
Space	$O(n^2)$ worst case	Up to n frames are stored, each of size n , useful for driving an animation.

Compiler Output / Test Cases:

```
frames = bubble_sort_with_frames([5, 1, 4, 2, 8])
assert frames[-1] == sorted([5, 1, 4, 2, 8])
assert frames[0] == [5, 1, 4, 2, 8]
assert len(frames) >= 2
print('All test cases passed!')
```

Question 4: Sorting Sensor Alerts by Severity

Approach: Implement both a plain bubble sort and an optimized version with the early-exit flag, then compare pass counts and comparisons on the same alert data to see how much the optimization helps for this small, often-partially-sorted stream.

Complexity Analysis:

Aspect	Complexity	Explanation
Plain Bubble Sort	$O(n^2)$ always	Always runs the full $n-1$ passes regardless of how sorted the data already is.
Optimized Bubble Sort	$O(n)$ best, $O(n^2)$ worst	Exits early once a pass makes no swaps, which helps when alerts arrive already close to sorted.

Compiler Output / Test Cases:

```
alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
r1, c1 = bubble_sort_plain(alerts)
r2, c2 = bubble_sort_optimized(alerts)
assert r1 == r2 == sorted(alerts) # both produce the same sorted result
assert c2 <= c1 # optimized never does more comparisons
print('Plain comparisons:', c1, 'Optimized comparisons:', c2)
print('All test cases passed!')
```

Question 5: Card Game Hand Sorting

Approach: Use optimized bubble sort after appending the new card. Because only one card is out of place, the algorithm should need very few passes (proportional to how far the new card must travel) compared to sorting the entire hand from an unsorted state.

Complexity Analysis:

Aspect	Complexity	Explanation
Nearly Sorted Hand	$\approx O(n)$	Only the newly added card needs to 'bubble' into place, so few passes are needed.
Fully Shuffled Hand	$O(n^2)$	A randomly shuffled hand requires close to the maximum number of passes and swaps.

Compiler Output / Test Cases:

```
hand = [2,4,6,8,9,11,13]
hand.append(7) # nearly sorted: one new card
final_hand, passes_incremental = bubble_sort_hand(hand)
assert final_hand == sorted(hand)
import random
shuffled = hand.copy()
random.shuffle(shuffled)
_, passes_full = bubble_sort_hand(shuffled)
assert passes_incremental <= passes_full # nearly-sorted needs no
more passes
print('All test cases passed!')
```

3. Insertion Sort

Question 1: Live Leaderboard Updates

Approach: Remove the updated player's old entry, then use the insertion-sort technique of shifting elements to find and insert the new score into its correct position in the already-sorted leaderboard, avoiding a full re-sort.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n)$	Only the shift needed to place the single new score is performed, instead of an $O(n \log n)$ or $O(n^2)$ full re-sort.
Space	$O(1)$ extra	Insertion happens via in-place shifting.

Compiler Output / Test Cases:

```
board = [980, 875, 760, 690, 500]
updated_board, shifts = insert_updated_score(board, 820)
assert updated_board == [980, 875, 820, 760, 690, 500]
board2, shifts2 = insert_updated_score(board, 100) # lowest score
assert board2[-1] == 100 and shifts2 == 0
print('All test cases passed!')
```

Question 2: Card Sorting While Playing

Approach: Maintain a hand list. Each time a new card is picked up, insert it into its correct position among the already-sorted cards by shifting larger cards one position to the right, exactly like insertion sort's inner loop.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n)$ per card, $O(n^2)$ total	Each new card may require shifting past several existing cards; over n cards this totals $O(n^2)$.
Space	$O(1)$ extra	Cards are shifted in place within the hand list.

Compiler Output / Test Cases:

```
hand = []
for card in [7, 2, 9, 4, 1]:
    hand = pick_up_card(hand, card)
assert hand == sorted([7, 2, 9, 4, 1])
assert pick_up_card([], 5) == [5] # first card picked up
print('All test cases passed!')
```

Question 3: Real-Time Stock Price Feed

Approach: Keep a running sorted list of prices. For each incoming price, use binary search (or a linear scan) to find its correct position, then shift subsequent elements right by one and insert it the core idea of insertion sort applied incrementally.

Complexity Analysis:

Aspect	Complexity	Explanation
Time (binary search variant)	$O(\log n)$ search + $O(n)$ shift	Finding the position is fast, but shifting array elements to make room is still $O(n)$ per insertion.
Overall for n prices	$O(n^2)$ worst case	Still quadratic overall due to shifting, but far better than re-sorting the whole list from scratch each time.

Compiler Output / Test Cases:

```
prices = []
for p in [102.5, 98.3, 105.1, 100.0, 97.8]:
    prices = insert_price(prices, p)
assert prices == sorted([102.5, 98.3, 105.1, 100.0, 97.8])
assert prices[0] == min(prices) and prices[-1] == max(prices) #
O(1) min/max lookup
print('All test cases passed!')
```

Question 4: Playlist Reordering by Recently Added

Approach: Append the new song's duration to the playlist and shift it leftward past any longer songs until it reaches its correct sorted position, exactly as the inner loop of insertion sort does for a single new element.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	$O(n)$	Only the new song is shifted into place among already-sorted songs.
Space	$O(1)$ extra	Insertion is performed via in-place shifting within the list.

Compiler Output / Test Cases:

```
playlist = [('Intro', 120), ('Chill Beat', 210), ('Long Jam', 340)]
updated_playlist = insert_song(playlist, ('Quick Track', 180))
durations = [s[1] for s in updated_playlist]
assert durations == sorted([120, 210, 340, 180])
assert ('Quick Track', 180) in updated_playlist
print('All test cases passed!')
```

Question 5: Nearly Sorted Sensor Log Correction

Approach: Run standard insertion sort on the log and count total shifts performed. Because only a few entries are out of place, most elements require zero or one shift, resulting in a shift count close to the best-case $O(n)$ rather than the worst-case $O(n^2)$.

Complexity Analysis:

Aspect	Complexity	Explanation
Nearly Sorted Data	$\approx O(n)$	Very few shifts are needed since most elements are already close to their correct position.
Randomly Shuffled Data	$O(n^2)$	Requires close to the maximum number of shifts,

		illustrating insertion sort's sensitivity to input order.
--	--	---

Compiler Output / Test Cases:

```
log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0]
sorted_log, shifts_nearly = insertion_sort_count_shifts(log)
assert sorted_log == sorted(log)
import random
shuffled_log = log.copy()
random.shuffle(shuffled_log)
shifts_random = insertion_sort_count_shifts(shuffled_log)
print('Nearly-sorted shifts:', shifts_nearly, '| Random shifts:',
shifts_random)
print('All test cases passed!')
```